Arduino Control Loop Instructions

Closed-loop altitude and descent-rate control for the cold-gas hopper on the vertical test stand

16.83 F26 · sprint 2/firmware-arduino · 2026-11-09

The hopper firmware now accepts guidance commands — *hover at 2 ft*, *descend at 0.1 m/s* — instead of only a manual valve percentage. This document covers what changed, which parameters to set before the first armed run, and how commands travel from the laptop to the vehicle.

> **STATUS**
>
> This firmware has **not been compiled or flown** — no Arduino toolchain was available on the machine where it was written. The control law was verified in a 1-DOF simulation, and every packet byte offset was checked programmatically against the dashboard decoder, but the build itself is unverified. Treat the first power-on as a bring-up, not a test point.

1 What changed

FILE	CHANGE
`control/gains.h`	`new` All tuning lives here. The only file to edit between test runs.
`control/controller.h/.cpp`	`new` The control law for every closed-loop mode.
`control/control.cpp`	`valve_demand()` now calls the controller; tracks loop `dt` and laser freshness.
`protocol.h` (both copies)	Commands gained `mode` + `setpoint`; telemetry gained seven controller fields.
`ground/bridge/bridge.cpp`	The E-stop override now also forces `MODE_MANUAL`.
`ground_station/index.html`	Mode picker, new plots, controller readout, extra CSV columns.

Unchanged on purpose: `safety.cpp` is still the only module that moves the valve, the SAFE / ARMED / FAILSAFE machine is untouched, and `MODE_MANUAL` is enum value `0` — so a sender that never sets the new fields behaves exactly as before.

> **PACKETS GREW — FLASH ALL THREE TOGETHER**
>
> Telemetry went 99 → 121 bytes and commands 17 → 22. The hopper, the ground ESP and the dashboard must all be updated in the same sitting. A mismatch is not subtle: the dashboard rejects every packet and logs a length warning, and the hopper ignores commands of the wrong size and drops to FAILSAFE after 150 ms.

2 How the control loop works

One equation serves every mode — a PD tracking a setpoint that is allowed to move:

```
demand = valve_hover + kp·(sp_z − z) + kd·(sp_vz − vz)
```

Hovering is that setpoint standing still. A rate command is the same setpoint moving at a constant speed. Because rate tracking keeps the altitude term, a descent that falls behind gets corrected rather than drifting further out, and switching modes is bumpless: the setpoint re-latches to wherever the vehicle actually is.

MODE	SETPOINT MEANS	BEHAVIOUR
`MANUAL`	—	Valve follows `valve_pct` directly. No feedback.
`HOVER`	metres above the pad	Ramps the setpoint at `sp_rate` to the commanded height, then holds. The dashboard takes feet and converts.
`VELOCITY`	m/s, up positive	Tracks a climb rate. `-0.1` descends at 0.1 m/s.
`LAND`	—	Descends at `land_rate`, then latches the valve shut below `land_cutoff`.

Three safeguards built into the loop

- **Altitude hold requires the laser.** The Kalman filter's `z` only stays true while the rangefinder keeps correcting it; without it the filter coasts on the IMU and drifts without bound. After 300 ms with no laser update (`EST_LASER_TIMEOUT_MS`) the loop is inhibited and the valve shut, flagged `CTRL_NO_EST`. Holding altitude on a free-running estimate would quietly fly away.
- **Touchdown latches.** Without the latch the vehicle settles at the cutoff, the error grows, the valve reopens, and it limit-cycles a few centimetres over the pad.
- **Setpoint re-latch.** Disarm, FAILSAFE and E-stop all drop back to Manual, so re-arming can never pick straight back up on a stale setpoint. In simulation this turned what would have been an 83 % saturated slam into a smooth 70 % near feedforward.

3 Parameters to adjust

CHECK THIS BEFORE YOU ARM · CONTROL.CPP LINES 24–25

```
// Which IMU axis points up, and its sign
#define IMU_UP_AXIS 2
#define IMU_UP_SIGN 1.0f
```

These predate the controller but matter far more now. The filter computes `a = accel_z − 9.81 − bias`, so `accel_z` must read **about +9.81 at rest**. Get the sign wrong and `vz` is garbage — and `kd = 22` acting on a garbage velocity drives the valve the wrong way, hard. Verify from the dashboard with the vehicle sitting still: `accel z ≈ +9.81` and `accel bias` settling near zero. Thirty seconds of checking.

PARAMETER	WHERE	CURRENT	HOW TO SET IT
`valve_hover`	gains.h	`50 %`	**Do this one first.** Hover in Manual mode and read the `valve_pct` that holds altitude.
laser offset	dashboard → Calibration	–	`z` is now the controlled variable, so its zero *is* the setpoint's zero.
`kp / kd`	gains.h	`18 / 22`	Already set. Fix `valve_hover` before touching these.
`SP_Z_MAX`	protocol.h	`1.00 m`	Must stay under both the laser's range and the stand's travel.
`valve_min / valve_max`	gains.h	`10 / 90 %`	Keep `valve_min` well under the hover opening — see the note below.
`sp_rate`	gains.h	`0.15 m/s`	How fast a new hover command ramps in, so a step does not ask for full valve on the first tick.
`land_rate / land_cutoff`	gains.h	`0.10 m/s / 0.05 m`	Descent rate, and the altitude where the valve latches shut.
`EST_LASER_TIMEOUT_MS`	gains.h	`300 ms`	Leave alone unless the laser's update rate changes.

VALVE_HOVER MATTERS MORE THAN EITHER GAIN

A PD has no integral term, so it cannot remove a constant error. The vehicle settles wherever the PD's output happens to equal the opening that holds it:

```
steady-state offset = (true hover opening − valve_hover) / kp
```

At `kp = 18` that is **5.6 cm of altitude error for every 1 %** this is wrong by, and the vehicle sits low when it is set too low. If a hover settles consistently short of its setpoint, raise `valve_hover` by `kp × offset` percent — do not reach for `kp`. Mass, supply pressure and feed temperature all move the true value, so re-check it whenever the vehicle changes.

WHY VALVE_MIN HAS A CEILING

`valve_min` above zero keeps the plenum from collapsing and means a descent is braked rather than a free fall. But at or above the hover opening the floor alone holds the vehicle up and *no descent is possible at all*. The current 10 % leaves plenty of margin: it brakes, it cannot hold.

Swapping gain sets

Change one line in `gains.h` and re-upload:

```
#define ACTIVE_GAINS GAINS_FLIGHT   // or GAINS_BENCH
```

`GAINS_FLIGHT` holds the known-good gains. `GAINS_BENCH` is a half-authority set (`kp 9`, `kd 11`, valve capped at 60 %, slow setpoint) for a first power-on on the tether — a sign error or a bad mounting axis then shows up as a drift you can watch rather than a slam into the stops. To add a set, copy a block and give it a new name; leave the flight set alone so there is always a known-good one to return to. The gains that actually flew are echoed in telemetry and recorded in the CSV, so a run always says which set produced it.

4 Sending commands from the laptop

The transport is unchanged. The dashboard streams commands at 50 Hz over USB to the ground ESP, which relays them to the hopper over ESP-NOW; telemetry returns the same way at 100 Hz. What is new is a **Mode** dropdown and its setpoint field.

```
laptop (Chrome, Web Serial) → USB → ground ESP → ESP-NOW → hopper
```

Bring-up order

1. Flash `hopper/hopper.ino` and `ground/ground.ino`, then hard-reload the dashboard. All three carry the new packet sizes.
2. Serve the dashboard over `http://localhost` in Chrome or Edge and press **Connect**. Web Serial will not work from a `file://` URL.
3. Confirm the IMU axis and sign (section 3), and that the `LASER` and `IMU` health chips are green.
4. Calibrate the laser height, so `z = 0` means on the pad.
5. **Arm in Manual** and find the valve percentage that holds a steady hover. That number is `valve_hover`.
6. Disarm. Put that value in `gains.h` and re-upload the hopper.
7. Mode → **Hover at altitude**, enter `2` ft. The page converts to metres and clamps to `SP_Z_MAX`.
8. Mode → **Climb / descend at rate**, slider to `-0.10` m/s.
9. Mode → **Land** to bring it down and latch the valve shut.

Safety behaviour while flying

- **Esc** is E-stop, and the physical switch on the ground board overrides whatever the dashboard sends — it now forces Manual as well as zero valve, so a closed-loop mode cannot keep commanding its own opening.
- Keep the tab in front. A hidden tab throttles its timers, commands stop arriving, and the hopper goes FAILSAFE after 150 ms.
- FAILSAFE latches: the valve is driven shut until you disarm and arm again. A reboot always comes up SAFE.

What to watch

The dashboard plots **Setpoint z** and **Valve demand** against `z` by default — the three traces that tell you whether the loop is tracking. The controller readout adds mode, both setpoints, the error, the raw pre-clamp demand, the live gains, and the `CTRL_*` flags (`SAT HIGH`, `SAT LOW`, `SP RAMP`, `NO EST`, `GND SHUT`). All of it lands in the recorded CSV, so a tuning run can be analysed after the fact.

Scripted commands instead of the dashboard

Write a 22-byte `cmd_t` with `mode` at byte 17 and `setpoint` as a little-endian `float32` at byte 18, framed for the USB link as:

> `0xAA 0x55 <len> <packet> <checksum = sum of packet bytes mod 256>`

Two constraints: the 150 ms link timeout means a script must sustain roughly 50 Hz or the hopper will FAILSAFE, and `arm` must be sent as `0` at least once after boot or after a FAILSAFE before the hopper will accept an arm request. Nothing else is printed on the ground board's serial port, so close the Arduino Serial Monitor before connecting.

5 Open items

- **Not compiled.** No Arduino toolchain was available; expect to fix build errors on first upload.
- `valve_hover = 50` **is a placeholder** derived from an assumed 0.40 kg mass. Measure it on the stand (step 5 above) before closing the loop.
- **No attitude estimate.** The filter assumes the IMU's chosen axis is vertical; any tilt shows up as a fake accelerometer bias. Acceptable on a constrained vertical stand, not off it.
- **Simulation used assumed plant values** — 0.40 kg, 5.87 N full-open thrust, 80 ms valve lag. The stability margins it showed are indicative, not measured.
